[Dung Vu] Threads & Synchronization

- **Section**: Linux System Programming
- **Topic**: Threads & Synchronization
- **Duration**: 4 working days (8 hours/day)

---

1. Thread

1.1. Thread Concepts

Threads are a mechanism that permits an application to perform multiple tasks concurrently.

On a multiprocessor system, multiple threads can execute parallel.

A process can contain multiple threads.

	Process	Thread
Definition	An instance of a program that is in execution	A unit of execution within a process
Memory	Has its own separate address space	Shares the address space (code, data, heap) of its parent process
Context switch	Slow (Full memory map/page table switch)	Fast (Only registers/stack pointer switch)
Isolation	A process crashing doesn't directly crash another	A thread crashing can crash the whole process
Information sharing	Difficult (Use IPC)	Easy (Use shared variables - global or heap)

1.2 Thread Identification

Each thread within a process is uniquely identified by a thread ID.

```
#include <pthread.h>

pthread_t pthread_self(void);

/* Returns the thread ID of the calling thread */
```

```
#include <pthread.h>

int pthread_equal(pthread_t t1, pthread_t t2);

/* Returns nonzero value if t1 and t2 are equal, otherwise 0 */
```

The `pthread_equal()` function is needed because the `pthread_t` data type must be treated as opaque data. Do not use `==` for `pthread_t`.

POSIX thread IDs are not the same as the thread IDs returned by the Linux-specific `gettid()` system call.

1.3. Thread Creation

```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                void *(*start)(void *), void *arg);

/* Returns 0 on success, or a positive error number on error */
```

- `thread`: a pointer to a buffer of type `pthread_t`
- `attr`: a pointer to a `pthread_attr_t` object that specifies atrributes for the new thread. Pass `NULL` for default attributes.
- `start`: a pointer to the startup function that the new thread begins executing. This function must take a single `void *` argument and return a `void *`
- `arg`: an argument of type `void *` passed directly to `start`
  - `NULL`: no argument
  - A global/heap variable: A single argument
  - A struct with multiples fields: Multiple arguments

When a new thread created, there is no guarantee of whether the calling thread or the new thread will be scheduled to run first.

1.4. Thread Termination

A thread terminates its execution in one of the following ways:

- Its `start` function performs a `return`
- It calls `pthread_exit()`
- It is canceled by another thread calling `pthread_cancel()`
- Any threads calls `exit()` or `main()` function performs a `return` => terminate all the threads in the process

```
#include <pthread.h>


void pthread_exit(void *retval);
```

If the **main** thread calls `pthread_exit()` instead of calling `exit()` or performing a `return`, then the other threads continue to execute.

1.5. Joining with a Terminated Thread

`pthread_join()` is how one thread waits for another thread to finish, and collects its result before its resources are cleaned up.

```
#include <pthread.h>


int pthread_join(pthread_t thread, void **retval);


/* Returns 0 on success, or a positive error number on error */
```

If a terminated thread never got joined or detached, it becomes a zombie thread.

Any thread in a process can use `pthread_join()` to join with any other thread in the process.

1.6. Detaching a Thread

Detaching a thread tells the system to automatically clean up the resource when it terminates.

Detached thread is not joinable.

```
#include <pthread.h>


int pthread_detach(pthread_t thread);
```

```
/* Returns 0 on success, or a positive error number on error */
```

A thread can detach itself by:

```
pthread_detach(pthread_self());
```

---

2. Concurrency

Concurrency is the ability of a system to handle multiple tasks during **overlapping time periods** (NOT *at the same time*).

2.1. Concurrency vs. Parallelism

concurrency_parallelism

Sequential: Tasks run one at a time, in order.

Concurrency: Multiple tasks make progress during **overlapping time periods**.

- Multiple tasks run on a single CPU core.
- Giving the illusion of "at the same time".

Parallelism: Multiple tasks execute at the same time, the exact same instant.

- Multiple tasks run on multiple cores/CPUs.
- True simultaneity.

2.2. Why Concurrency is hard

Race Conditions

- Occurs when the outcome of a program depends on the timing of operations by multiple threads.
- Example: if two threads are trying to update a shared variable without proper synchronization, the final value of the variable can be unpredictable.

Deadlocks

- Occurs when 2 or more threads are waiting for each other to release resources => resulting in a situation where none of the thread can proceed

Resource Management

- Multiple threads accessing the same memory location can lead to data corruption without proper synchronization.

Synchronization Overhead

- Synchronization mechanisms, such as locks, semaphores, and mutexes, introduce overhead that can impact performance.

Complexity of Debugging

- Debugging concurrent programs is inherently more complex than debugging sequential programs.
- Issues such as race conditions and deadlocks can be difficult to reproduce and diagnose.

---

3. Real-world Case Study: the Mars Pathfinder priority-inversion bug

Priority Inversion: A high priority task can become blocked by a lower priority task indefinitely, if lower priority task locks access to resources shared by both tasks.

NASA Mars Pathfinder Bug:

- Occured on NASA's Mars Pathfinder mission in 1997.
- The rover had several tasks running at different priority levels:
  - *Low-priority* task: collecting data from the rover's instruments.
  - *High-priority* task: handling critical operations that needed to run frequently.
  - *Medium-priority* task: performed communications and other background work.
- At one point:
  - *Low-priority* task acquired a mutex to collect data.
  - Before *low-priority* task could release the resource, *high-priority* task needed it.
  - Normally *low-priority* would finish quickly, but *medium-priority* task preempted it and hogged the CPU.
  - *High-priority* was stuck waiting for a *low-priority* that was unable to run.

mars_pathfinder_bug

=> The engineers fixed the issue using a mechanism called **priority inheritance**:

- When a *low-priority* task holds a resource needed by a *high-priority* task, the system temporarily boosts the *low-priority* task's priority.

4. Thread Synchronization

4.1. Atomic & Non-atomic

Non-atomic operation

- Consists of multiple underlying steps.
- Can be interrupted midway at any assembly instruction.

Atomic operation

- An operation that executes as a single, indivisible unit
- It either completes fully or doesn't start at all.
- Can never be interrupted or observed in a partially complete state.

4.2 Critical Section

Critical section

- A section of code that accesses a shared resource.
- Its execution should be atomic.

An example about Critical section and Non-atomic operation in Lab 1

4.3. Mutex locks

A mutex locks ensures that only one thread at a time can hold the lock and execute the critical section protected by it. Any other thread trying to acquire the same mutex will block until the thread holding it releases it.

Statically Allocating a Mutex:

```
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
```

- Used for initializing a statically allocated mutex with default attributes

Dynamically Initializing a Mutex:

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

Locking and Unlocking a Mutex:

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);
/* Attempt to lock a mutex instead of blocking */
```

---

5. Reader-writer locks

Reader-writer locks let multiple readers access shared data concurrently, but writers get exclusive access. Useful when reads vastly outnumber writes.

Lock for reading:

```
int pthread_rwlock_rdlock(pthread_rwlock_t *rwlock);
```

Lock for writing:

```
int pthread_rwlock_wrlock(pthread_rwlock_t *rwlock)
```

Release for both:

```
int pthread_rwlock_unlock(pthread_rwlock_t *rwlock)
```

Non-blocking variants:

```
int pthread_rwlock_tryrdlock(pthread_rwlock_t *rwlock);
int pthread_rwlock_trywrlock(pthread_rwlock_t *rwlock);
```

---

6. Reentrancy and Thread-Specific Data

6.1. Reentrancy

Reentrancy is a property of a function which can be interrupted at any point during its execution and then safely called again ("re-entered") before its previous invocations complete execution.

A reentrant function achieves thread safety without the use of mutexes.

A reentrant function:

- Avoids the use of global and static variables.
- Does not use mutex locks.
- Any information that must be returned to the caller is stored in buffers allocated by the caller.

Example:

```
/* A non-reentrant example */
/* [The function depends on global variable i] */
int i;

/* Both fun1() and fun2() are not reentrant */

/* fun1() is NOT reentrant because it uses global variable i */
int fun1()
{
    return i * 5;
}

/* fun2() is NOT reentrant because it calls a non-reentrant function */
int fun2()
{
   return fun1() * 5;
}
```

```
/* Both fun1() and fun2() are reentrant */
int fun1(int i)
{
    return i * 5;
}

int fun2(int i)
{
   return fun1(i) * 5;
}
```

6.2. Thread-Specific Data

Thread-specific data allows a function to maintain a separate copy of a variable for each thread that calls the function.

Each thread’s variable continues to exist between the thread’s invocations of the function.

Thread-specific data items are differentiated by **keys**

```
int pthread_key_create(pthread_key_t *key, void (*destructor)(void *));

/* Returns 0 on success, or a positive error number on error */
```

```
int pthread_setspecific(pthread_key_t key, const void *value);

/* Returns 0 on success, or a positive error number on error */

void *pthread_getspecific(pthread_key_t key);

/* Returns pointer, or NULL if no thread-specific data is associated with key */
```

Example:

```
#include <stdio.h>
#include <pthread.h>
#include <stdint.h>

pthread_key_t counter_key;
```

```
void print_counter()
{
    /* Retrieve the value from thread-local storage */
    int counter = (int)(intptr_t)pthread_getspecific(counter_key);

    printf("Thread %lu: counter = %d\n",
            pthread_self(), counter);
}

void add_counter(int add_value)
{
    /* Retrieve the value from thread-local storage */
    int counter = (int)(intptr_t)pthread_getspecific(counter_key);

    /* Update the counter */
    counter += add_value;

    /* Store the value in thread-local storage */
    pthread_setspecific(counter_key, (void *)(intptr_t)counter);
}

void *thread_func(void *arg)
{
    pthread_setspecific(counter_key, (void *)(intptr_t)0);

    for(int i = 1; i <= 10; i++)
    {
        add_counter(1);
        print_counter();
    }

    return NULL;
}

int main(void)
{
    pthread_t t1, t2;

    /* Create the thread-specific data key */
```

```
    pthread_key_create(&counter_key, NULL);

    /* Thread creations*/
    pthread_create(&t1, NULL, thread_func, NULL);
    pthread_create(&t2, NULL, thread_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    /* Delete the key */
    pthread_key_delete(counter_key);

    return 0;
}
```

Output:

```
Thread 127454999996096: counter = 1
Thread 127454999996096: counter = 2
Thread 127454999996096: counter = 3
Thread 127454999996096: counter = 4
Thread 127454999996096: counter = 5
Thread 127454999996096: counter = 6
Thread 127455008388800: counter = 1
Thread 127455008388800: counter = 2
Thread 127455008388800: counter = 3
Thread 127455008388800: counter = 4
Thread 127455008388800: counter = 5
Thread 127455008388800: counter = 6
Thread 127455008388800: counter = 7
Thread 127455008388800: counter = 8
Thread 127455008388800: counter = 9
Thread 127455008388800: counter = 10
Thread 127454999996096: counter = 7
Thread 127454999996096: counter = 8
Thread 127454999996096: counter = 9
Thread 127454999996096: counter = 10
```

We can see that, although 2 seperate threads (TIDs `...96` and `...00`) using the same key, each thread maintains a completely independent counter value.

7. Threads and Signals, Threads and fork, Threads and I/O

7.1. Threads and Signals

Signals action and dispositions are process-wide.

If any stop or terminate signal is delivered to any thread in the process, all of the threads in the process are ***stopped*** or ***terminated***. All threads in a process share the same disposition for each signal.

7.2. Threads and fork

When a multithreaded process calls `fork()`:

- Only the calling thread is replicated in the child process.
- All of the other threads vanish in the child, without executing thread-specific data destructors or cleanup handlers.
- Lead to various problems:
  - Locked mutexes cause deadlock:
    Mutexes held by other threads during `fork()` remain locked in the child process.
    Since their owners don't exist in the child, acquiring them causes a permanent deadlock.
  - Data inconsistency:
    Global data structures can be corrupted if a thread was interrupted mid-update.
  - Memory leaks:
    Uncleaned resources left behind by vanished threads.

Example: Recreation of deadlock in child process

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/wait.h>

pthread_mutex_t mutex;
```

```
void *thread1(void* arg) /* thread 1 */
{
    char thread_name[] = "Thread 1";

    pthread_mutex_lock(&mutex);
    printf("[%s] LOCKED MUTEX.\n", thread_name);

    sleep(2); /* Simulate some work */

    pthread_mutex_unlock(&mutex);
    printf("[%s] UNLOCKED MUTEX.\n", thread_name);

    return NULL;
}

void *thread2(void* arg) /* thread 2 */
{
    char thread_name[] = "Thread 2";
    char process_name[20];

    sleep(1); /* Wait for thread 1 to acquire the mutex */
    int pid = fork();

    if (pid == 0) /* Child process */
        snprintf(process_name, sizeof(process_name), "Child Process");
    else if (pid > 0) /* Parent process */
        snprintf(process_name, sizeof(process_name), "Parent Process");

    /* Attempt to lock the mutex */
    while(pthread_mutex_trylock(&mutex) != 0)
    {
        printf("[%s][%s] Failed to lock mutex.\n", thread_name, process_name);
        sleep(1);
    }
    printf("[%s][%s] Locked mutex.\n", thread_name, process_name);

    pthread_mutex_unlock(&mutex);
    printf("[%s][%s] Unlocked mutex.\n", thread_name, process_name);
```

```
    if (pid == 0) /* Child process */
        exit(0);
    else if (pid > 0) /* Parent process */
        wait(NULL);

    return NULL;
}

int main()
{
    pthread_mutex_init(&mutex,NULL);

    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

Output:

```
[Thread 1] LOCKED MUTEX.
[Thread 2][Parent Process] Failed to lock mutex.
[Thread 2][Child Process] Failed to lock mutex.
[Thread 1] UNLOCKED MUTEX.
[Thread 2][Parent Process] Locked mutex.
[Thread 2][Parent Process] Unlocked mutex.
[Thread 2][Child Process] Failed to lock mutex.
[Thread 2][Child Process] Failed to lock mutex.
[Thread 2][Child Process] Failed to lock mutex.
[Thread 2][Child Process] Failed to lock mutex.
[Thread 2][Child Process] Failed to lock mutex.
[Thread 2][Child Process] Failed to lock mutex.
...
```

> Child process: mutex remains locked after `fork()`

- Parent process:
  - `thread1` locks the mutex first.
  - 1 sec later, `thread2` calls `fork()`.
  - 2 sec later, `thread1` releases the mutex.
  - `thread2` (parent) locks and unlocks the mutex normally
- Child process:
  - The child is created when the mutex is LOCKED.
  - `fork()` only copies the calling thread. `thread1` is not created in the child process.
  - Therefore, no thread in the child can release the mutex.

  => The mutex in the child remains locked indefinitely
  => PERMANENT BLOCKING

---

8. Condition Variables and Barriers

8.1. Condition Variables

A condition variable allows one thread to inform other threads about changes in the state of a shared variable and allows the other threads to wait (block) for such notification.

A condition variable is always used in conjunction with a mutex.

The mutex provides mutual exclusion for accessing the shared variable, while the condition variable is used to signal changes in the variable's state.

Statically Allocated Condition Variables:

```
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
```

Dynamically Allocated Condition Variables:

```
int pthread_cond_init(pthread_cond_t *cond, const pthread_condattr_t *attr);
int pthread_cond_destroy(pthread_cond_t *cond);
```

Signaling and Waiting on Condition Variables:

Signaling:

```
int pthread_cond_signal(pthread_cond_t *cond);
```

- at least one of the blocked threads is woken up

```
int pthread_cond_broadcast(pthread_cond_t *cond);
```

- all blocked threads are woken up

Waiting:

```
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
```

- performs the following steps:
  1. Unlock the currently held mutex
  2. Block the calling thread until the condition variable is signaled
  3. Relock the mutex
- blocks a thread until the condition variable `cond` is signaled
- must be governed by a `while` loop rather than an `if`, because *other threads may be woken up first* and *spurious wake-ups can occur*

```
int pthread_cond_timedwait(pthread_cond_t *cond, pthread_mutex_t *mutex,
                           const struct timespec *abstime);
```

- is the same as `pthread_cond_wait()`, except that the `abstime` argument specifies an upper limit on the time that the thread will sleep while waiting for the condition variable to be signaled.

---

9. Semaphores

9.1. Concepts

A semaphore is a non-negative integer variable that is shared between various threads.

Semaphore works upon signaling mechanism, in this a thread can be signaled by another thread.

Two types:

- Binary semaphore: 0 or 1, used for signaling
- Counting semaphore: tracks N available instances of a resource

Semaphore has no priority inheritance.

9.2. Semaphores vs. Mutexes

	Mutex	Semaphore
Concept	Locking mechanism	Signaling mechanism
Ownership	Same thread locks/unlocks	Any thread can give/take
Purpose	Protecting a shared resource	Task synchronization / event notification / counting resources
Priority inheritance	Usually yes	No

9.3. POSIX Semaphores

There are 2 types of POSIX Semaphores:

- Named semaphore:
  - Has a name
  - Calling `sem_open()` with the same name, unrelated processes can access the same semaphore.
- Unnamed semaphore:
  - Doesn't have a name
  - Can be shared between processes or between a group of threads
  - When shared between processes, must reside in a region of shared memory (System V, POSIX, mmap())
  - When shared between threads, reside in an area of memory shared by the threads (heap/global variable)

Named Semaphore

Opening a Named Semaphore: creates and opens a new named semaphore or opens an existing semaphore

```
#include <fcntl.h> /* Defines O_* constants */
#include <sys/stat.h> /* Defines mode constants */
#include <semaphore.h>

sem_t *sem_open(const char *name, int oflag, ...
/* mode_t mode, unsigned int value */ );

/* Returns pointer to semaphore on success, or SEM_FAILED on error */
```

Closing a Semaphore: terminates the association of the semaphore

- Releases any resources of the semaphore for this process
- Decreases the count of processes referencing the semaphore
- Closing a semaphore does not delete it

```
int sem_close(sem_t *sem);


/* Returns 0 on success, or –1 on error */
```

Removing a Named Semaphore: removes the semaphore identified by `name` and marks the semaphore to be destroyed once all processes cease using it

```
int sem_unlink(const char *name);


/* Returns 0 on success, or –1 on error */
```

Unnamed Semaphore

Initializing an Unnamed Semaphore:

```
int sem_init(sem_t *sem, int pshared, unsigned int value);


/* Returns 0 on success, or –1 on error */
```

- `pshared`:
  - 0: shared between threads of the calling process
  - nonzero: shared between processes
- `value`: initial value of the semaphore

Destroying an Unnamed Semaphore:

```
int sem_destroy(sem_t *sem);


/* Returns 0 on success, or –1 on error */
```

- It is safe to destroy a sema- phore only if no processes or threads are waiting on it.

Semaphore Operations

Waiting on a Semaphore: Decreases the value of the semaphore `sem` by 1

```
int sem_wait(sem_t *sem);
int sem_trywait(sem_t *sem);
int sem_timedwait(sem_t *sem, const struct timespec *abs_timeout);

/* Returns 0 on success, or –1 on error */
```

Posting a Semaphore: Increases the value of the semaphore `sem` by 1

```
int sem_post(sem_t *sem);

/* Returns 0 on success, or –1 on error */
```

Retrieving the Current Value of a Semaphore: returns the current value of the semaphore `sem` in the `int` pointed to by `sval`.

```
int sem_getvalue(sem_t *sem, int *sval);

/* Returns 0 on success, or –1 on error */
```

---

10. Lock implementation

10.1. Spinlock

A spinlock is a low-level synchronization mechanism that protects a critical section by “spinning” in a tight loop until the lock becomes available.

The spin loop is built on an atomic CAS (compare and swap) instruction:

```
while (!compare_and_swap(&lock, 0, 1)) {
    // failed, lock still held – try again
}
```

`compare_and_swap(address, expected, new)` atomically:

1. Reads the value at `address`
2. Compares it to `expected`
3. If they match → writes `new`, returns success
4. If not → leaves it unchanged, returns failure

10.2. Blocking Locks

A blocking lock is a synchronization mechanism where a thread that fails to acquire the lock is put to sleep (removed from the CPU run queue) instead of busy-waiting.

Including:

- Mutex
- Semaphore
- File lock
- Condition variable

	Spinlock	**Blocking lock**
Behavior	Repeatedly checking the lock	Putting thread to sleep
Space	User space	Kernel space
Pros	Fast lock acquire/release	Consumes no CPU while sleeping
Cons	Wastes CPU cycles	More expensive context switch
Usages	Very short critical sections	Longer critical sections, uncertain/long wait times

10.3. Futex

A futex (fast userspace mutex) is a Linux mechanism that combines the two approaches above (Spinlock + Blocking locks)

The futex is a 32-bit `int` in user space.

Lock flow:

- Thread attempts atomic CAS in user space.
  - Success -> enters critical section, no syscall.
  - Failure -> calls `futex(FUTEX_WAIT, ...)`

Unlock flow

- Thread clears the lock value in user space.
- Checks a waiter flag tracked in the lock
  - No waiters → done, no syscall
  - Waiters present → `calls futex(FUTEX_WAKE, ...)`, kernel wakes one thread off the wait queue.

---

11. Deadlock

When more than one threads is locking the same set of mutexes, deadlock situations can arise.

```
Thread A
1. pthread_mutex_lock(mutex1);
2. pthread_mutex_lock(mutex2);
blocks

Thread B
1. pthread_mutex_lock(mutex2);
2. pthread_mutex_lock(mutex1);
blocks
```

> These 2 threads wait for each other to release the lock.

Ways to avoid deadlock:

- Define a mutex hierarchy: When using a the same set of mutexes, threads should always lock them in the same order:

  ```
  Thread A
  1. pthread_mutex_lock(mutex1);
  2. pthread_mutex_lock(mutex2);

  Thread B
  1. pthread_mutex_lock(mutex1);
  2. pthread_mutex_lock(mutex2);

  Lock mutex1 then mutex2
  ```

- Use `pthread_mutex_trylock()` to avoid blocking:

  ```
  Thread A
  1. pthread_mutex_lock(mutex1);
  2. pthread_mutex_lock(mutex2);

  Thread B
  1. pthread_mutex_trylock(mutex2);
  ```

```
2. pthread_mutex_trylock(mutex1);
```

---

12. Lab

Link to lab

Lab 1

Create and join multiple threads sharing a counter; observe a race condition on the unprotected shared data

```
#include <stdio.h>
#include <pthread.h>

long counter = 0; /* global variable */

void* increment(void* arg) /* thread function */
{
    for (int i = 0; i < 1000000; i++)
    {
        counter++;
    }
    return NULL;
}

int main()
{
    pthread_t t1, t2;

    /* Create two threads that do the incrementing */
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    /* Wait for both threads to complete */
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
```

```
    printf("Result: %ld (expected: 2000000)\n", counter);
    return 0;
}
```

Output:

```
$ ./lab_1
Result: 1345282 (expected: 2000000)
$ ./lab_1
Result: 1003811 (expected: 2000000)
$ ./lab_1
Result: 1098357 (expected: 2000000)
```

The expected result is 2000000. However, the actual result is lower than the expectation, and varies unpredictably between runs due to race condition.

The `counter++` operation is non-atomic, it actually consists of 3 CPU instructions (Read-Modify-Write):

```
movq    counter(%rip), %rax ; 1. Read: Load counter data from memory into rax register
addq    $1, %rax            ; 2. Modify: Add 1 into rax
movq    %rax, counter(%rip) ; 3. Write: Store rax data back to counter variable
```

And the fact that `counter` is a shared resource makes `counter++` a critical section.

When those two threads run concurrently, instructions of 2 `++` operators can collide.

Example of 2 `counter++` instructions colliding:

Assuming that counter initially equals 5

Time	Thread 1 (eax_T1)	Thread 2 (eax_T2)	counter (RAM)
t1	`movq counter,%rax` → eax_T1 = 5		5
t2		`movq counter,%rax` → eax_T2 = 5	5
t3	`addq 1, %rax` → eax_T1 = 6		5
t4		`addq $1, %rax` → eax_T2 = 6	5
t5	`movq %rax, counter` → writes 6		**6**

Time	Thread 1 (eax_T1)	Thread 2 (eax_T2)	counter (RAM)
t6		`movq %rax, counter` → writes 6	**6**

The counter result is 6 instead of 7.

Lab 2

Fix the race condition using a mutex lock; verify correctness under load

We fixed the race condition by protecting the critical section (`counter++`) using a mutex lock

```
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t mutex;
long counter = 0; /* global variable */

void* increment(void* arg) /* thread function */
{
    for (int i = 0; i < 1000000; i++)
    {
        pthread_mutex_lock(&mutex);
        counter++;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

int main()
{
    pthread_mutex_init(&mutex, NULL);
    pthread_t t1, t2;

    /* Create two threads that do the incrementing */
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    /* Wait for both threads to complete */
    pthread_join(t1, NULL);
```

```
    pthread_join(t2, NULL);

    printf("Result: %ld (expected: 2000000)\n", counter);

    pthread_mutex_destroy(&mutex);
    return 0;
}
```

Output:

```
$ ./lab_2
Result: 2000000 (expected: 2000000)
```

Lab 3

Test the reentrancy (thread-safety) of a self-written function and fix it if it is not reentrant

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <string.h>

/* Non-reentrant function */
char *get_message(const char *name)
{
    static char buffer[100]; /* Non-reentrant since using a static buffer */
    snprintf(buffer, sizeof(buffer), "Hi, I'm %s", name);
    sleep(1); /* Make the race easier to observe */

    return buffer;
}

void *thread_func(void *arg)
{
    const char *name = (const char *)arg;
    char *msg = get_message(name);
    printf("[%s thread] %s\n", name, msg);

    return NULL;
```

```
}

int main(void)
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread_func, "Alice");
    pthread_create(&t2, NULL, thread_func, "Bob");

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

Output:

```
[Alice thread] Hi, I'm Bob
[Bob thread] Hi, I'm Bob
```

=> Thread Alice returns the same shared buffer after Thread Bob has modified it.

Let the caller provide the buffer instead:

```
char *get_message(const char *name, char *buffer, size_t buffer_size) /* Reentrant function */
{
    snprintf(buffer, buffer_size, "Hi, I'm %s", name);
    sleep(1);

    return buffer;
}

void *thread_func(void *arg)
{
    const char *name = (const char *)arg;

    char buffer[100]; /* The thread provides buffer */
    char *msg = get_message(name, buffer, sizeof(buffer));
    printf("[%s thread] %s\n", name, msg);

    return NULL;
```

```
}
```

Output:

```
[Alice thread] Hi, I'm Alice
[Bob thread] Hi, I'm Bob
```

The program now work as intended.

Lab 4

Implement a producer-consumer program using a mutex and a condition variable

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t mutex;
pthread_cond_t cond;
long counter = 0;

void* producer(void* arg)
{
    while (1)
    {
        pthread_mutex_lock(&mutex);
        counter += 1;
        printf("[Producer] produced 1, counter = %ld\n", counter);
        pthread_cond_signal(&cond);
        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
    return NULL;
}

void* consumer(void* arg)
{
    while (1)
    {
        pthread_mutex_lock(&mutex);
```

```
        while (counter < 4) /* wait for the producer to produce */
            pthread_cond_wait(&cond, &mutex);

        counter -= 4; /* consume items */
        printf("[Consumer] consumed 4, counter = %ld\n", counter);
        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
    return NULL;
}

int main()
{
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&cond, NULL);

    pthread_t t1, t2;

    pthread_create(&t1, NULL, producer, NULL);
    pthread_create(&t2, NULL, consumer, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&cond);
    return 0;
}
```

Output:

```
$ ./lab_4
[Producer] produced 1, counter = 1
[Producer] produced 1, counter = 2
[Producer] produced 1, counter = 3
[Producer] produced 1, counter = 4
[Consumer] consumed 4, counter = 0
[Producer] produced 1, counter = 1
[Producer] produced 1, counter = 2
[Producer] produced 1, counter = 3
```

```
[Producer] produced 1, counter = 4
[Consumer] consumed 4, counter = 0
[Producer] produced 1, counter = 1
[Producer] produced 1, counter = 2
[Producer] produced 1, counter = 3
[Producer] produced 1, counter = 4
[Consumer] consumed 4, counter = 0
[Producer] produced 1, counter = 1
[Producer] produced 1, counter = 2
[Producer] produced 1, counter = 3
[Producer] produced 1, counter = 4
[Consumer] consumed 4, counter = 0
```

Producer and consumer's critical section are protected by a mutex. Consumer, which consuming 4 items, uses a conditional variable in order to wait for the producer creating >= 4 items.

Lab 5

Reproduce a deadlock between two threads acquiring two locks in opposite order, then fix it via consistent lock ordering

In this lab, we got:

- 2 mutex: `mutex1` and `mutex2`
- `thread_a` lock order: lock `mutex2` then `mutex1`
- `thread_b` lock order: lock `mutex1` then `mutex2`
- Both `thread_a` and `thread_b` change increase `counter` by 1.
- `thread_c`: track changes of `counter`, exit if `counter` hasn't be changed after 1 second.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t mutex1;
pthread_mutex_t mutex2;
long counter = 0; /* global variable */

void* thread_a(void* arg)
{
    while (1)
```

```
    {
        pthread_mutex_lock(&mutex2);
        pthread_mutex_lock(&mutex1);
        counter++;
        printf("[Thread A] counter = %ld\n", counter);
        pthread_mutex_unlock(&mutex2);
        pthread_mutex_unlock(&mutex1);
        usleep(100000);
    }
    return NULL;
}

void* thread_b(void* arg)
{
    while (1)
    {
        pthread_mutex_lock(&mutex1);
        pthread_mutex_lock(&mutex2);
        counter++;
        printf("[Thread B] counter = %ld\n", counter);
        pthread_mutex_unlock(&mutex1);
        pthread_mutex_unlock(&mutex2);
        usleep(100000);
    }
    return NULL;
}

void* thread_c(void* arg) /* deadlock timeout check */
{
    while (1)
    {
        long old_counter = counter;
        sleep(1);
        if (old_counter == counter)
        {
            printf("[Thread C] Deadlock detected!\n");
            exit(-1);
        }
    }
    return NULL;
```

```
}

int main()
{
    pthread_mutex_init(&mutex1, NULL);
    pthread_mutex_init(&mutex2, NULL);

    pthread_t t1, t2, t3;

    pthread_create(&t1, NULL, thread_a, NULL);
    pthread_create(&t2, NULL, thread_b, NULL);
    pthread_create(&t3, NULL, thread_c, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&mutex1);
    pthread_mutex_destroy(&mutex2);
    return 0;
}
```

Output of sevaral tries:

```
[Thread A] counter = 1
[Thread B] counter = 2
[Thread C] Deadlock detected!
```

```
[Thread A] counter = 1
[Thread B] counter = 2
[Thread A] counter = 3
[Thread B] counter = 4
[Thread C] Deadlock detected!
```

```
...
[Thread B] counter = 6
[Thread A] counter = 7
[Thread B] counter = 8
[Thread C] Deadlock detected!
```

Fix:

2 threads follow the same lock order: lock `mutex1` then `mutex2`

```
void* thread_a(void* arg)
{
    while (1)
    {
        pthread_mutex_lock(&mutex1);
        pthread_mutex_lock(&mutex2);
        counter++;
        printf("[Thread A] counter = %ld\n", counter);
        pthread_mutex_unlock(&mutex2);
        pthread_mutex_unlock(&mutex1);
        usleep(100000);
    }
    return NULL;
}

void* thread_b(void* arg)
{
    while (1)
    {
        pthread_mutex_lock(&mutex1);
        pthread_mutex_lock(&mutex2);
        counter++;
        printf("[Thread B] counter = %ld\n", counter);
        pthread_mutex_unlock(&mutex2);
        pthread_mutex_unlock(&mutex1);
        usleep(100000);
    }
    return NULL;
}
```

Output:

```
...
[Thread B] counter = 1009
[Thread A] counter = 1010
[Thread B] counter = 1011
[Thread A] counter = 1012
[Thread A] counter = 1013
```

```
[Thread B] counter = 1014
[Thread A] counter = 1015
[Thread B] counter = 1016
[Thread A] counter = 1017
[Thread B] counter = 1018
[Thread A] counter = 1019
[Thread B] counter = 1020
...
```

> No longer getting a deadlock!

Revision #15
Created 2026-08-28 07:18:39 UTC by Dung Vu
Updated 2026-09-21 02:26:46 UTC by Dung Vu